week03_data_split_and_eda

February 12, 2026

[1]:
```
import os, subprocess

REPO_URL = "https://github.com/tongyuguo/HelpHerInvest.git"
REPO_DIR = "HelpHerInvest"
data_dir = os.path.join(REPO_DIR, "Data")

def clone_or_pull():
    if os.path.isdir(os.path.join(REPO_DIR, ".git")):
        subprocess.run(["git", "-C", REPO_DIR, "pull"])
    else:
        subprocess.run(["git", "clone", REPO_URL])

clone_or_pull()
```

```
Updating eae5f64..009b99e
Fast-forward
 …/Week 3 noteboot edit/week03_NLP_EDA.ipynb      | 3655 +++++++++
 …/week03_data_split_and_eda.ipynb                | 8410 +++++++++++++++++++++
 …/week03_data_split_and_technitical_EDA1.ipynb   | 1647 ++++
 …/Tongyu Playground/nlp data cleaning.ipynb      |   27 +-
 4 files changed, 13727 insertions(+), 12 deletions(-)
 create mode 100644 Playground/Week 04 Make data model ready/Tongyu
Playground/Week 3 noteboot edit/week03_NLP_EDA.ipynb
 create mode 100644 Playground/Week 04 Make data model ready/Tongyu
Playground/Week 3 noteboot edit/week03_data_split_and_eda.ipynb
 create mode 100644 Playground/Week 04 Make data model ready/Tongyu
Playground/Week 3 noteboot edit/week03_data_split_and_technitical_EDA1.ipynb

From https://github.com/tongyuguo/HelpHerInvest
   eae5f64..009b99e  main       -> origin/main
```

[2]:
```
import numpy as np
import pandas as pd
#%pip install yfinance
import yfinance as yf
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
```

```python
warnings.filterwarnings("ignore")

from datetime import datetime

import re

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.neighbors import NearestNeighbors

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_colwidth", 180)
sns.set_theme(style="whitegrid")
```

[3]:
```python
import zipfile

zip_path = "HelpHerInvest/Data/final_dataset.csv.zip"

with zipfile.ZipFile(zip_path) as z:
    df = pd.read_csv(z.open("final_dataset.csv"))

df.head()
```

[3]:
```
         Date Ticker    mom_1m  mom_3m  mom_6m  mom_12m  mom_12m_ex_1m  \
0  2010-02-28   NVDA  0.587193     NaN     NaN      NaN            NaN
1  2010-02-28  GOOGL -0.705474     NaN     NaN      NaN            NaN
2  2010-02-28   AAPL  0.868995     NaN     NaN      NaN            NaN
3  2010-02-28   MSFT -0.085788     NaN     NaN      NaN            NaN
4  2010-02-28   AMZN -1.808631     NaN     NaN      NaN            NaN

   rel_3m_spy  rel_6m_spy  rel_12m_spy  vol_3m  vol_6m  drawdown_6m  \
0         NaN         NaN          NaN     NaN     NaN          NaN
1         NaN         NaN          NaN     NaN     NaN          NaN
2         NaN         NaN          NaN     NaN     NaN          NaN
3         NaN         NaN          NaN     NaN     NaN          NaN
4         NaN         NaN          NaN     NaN     NaN          NaN

   drawdown_12m  pct_above_200dma  adj_close  fwd_excess  fwd_return
0           NaN               NaN   0.371318   -0.383499   -0.424074
1           NaN               NaN  13.084321   -0.105173   -0.145748
2           NaN               NaN   6.136767    0.228631    0.188056
3           NaN               NaN  21.526321   -0.132719   -0.173294
4           NaN               NaN   5.920000    0.094883    0.054307
```

[4]:
```python
# train/val/test split
idx = len(df.index)
print(f"Dataset loaded with {idx} rows.")
```

```
train_num = int(idx * 0.7)
val_num = int(idx * 0.15)
test_num = idx - train_num - val_num
print(f"Train set: {train_num} rows"
      f"\nValidation set: {val_num} rows"
      f"\nTest set: {test_num} rows")

training_data = df.iloc[:train_num]
training_data.tail()
```

```
Dataset loaded with 9018 rows.
Train set: 6312 rows
Validation set: 1352 rows
Test set: 1354 rows
```

[4]:

```
            Date Ticker    mom_1m    mom_3m    mom_6m   mom_12m  \
6307  2021-06-30     KO -0.664909 -0.725725 -1.178227 -0.658839
6308  2021-06-30     MS -0.312631  0.928687  1.010281  1.192557
6309  2021-06-30    AZN  0.251339  1.117750  0.222708 -0.836243
6310  2021-06-30   CSCO -0.412172 -0.747593  0.079779 -0.862740
6311  2021-06-30    CAT -1.992531 -1.725354  0.110645  0.704804

      mom_12m_ex_1m  rel_3m_spy  rel_6m_spy  rel_12m_spy    vol_3m    vol_6m  \
6307      -0.543735   -0.725725   -1.178227    -0.658839 -1.460881 -1.031446
6308       1.301193    0.928687    1.010281     1.192557  0.227042 -0.099618
6309      -0.916816    1.117750    0.222708    -0.836243  1.017971  0.052143
6310      -0.812849   -0.747593    0.079779    -0.862740 -0.814890 -0.779071
6311       1.364474   -1.725354    0.110645     0.704804 -0.271047 -0.169314

      drawdown_6m  drawdown_12m  pct_above_200dma   adj_close  fwd_excess  \
6307     0.215115       0.27088         -0.963440   47.294609   -0.005968
6308     0.514905       0.55748          1.392062   78.470230   -0.031146
6309     0.514905       0.55748          0.068075  110.269676   -0.154453
6310     0.514905       0.55748          0.012837   46.186298    0.095101
6311    -1.572799      -1.43837         -0.509397  199.377823   -0.156697

      fwd_return
6307    0.111061
6308    0.085883
6309   -0.037424
6310    0.212130
6311   -0.039668
```

[5]:

```
# Perform basic EDA
count_nas = training_data.isna().sum()
print("Missing values per column:")
print(count_nas)
```

```python
# Summary statistics
training_data.describe()
```

```
Missing values per column:
Date                  0
Ticker                0
mom_1m                0
mom_3m               96
mom_6m              240
mom_12m             526
mom_12m_ex_1m       526
rel_3m_spy           96
rel_6m_spy          240
rel_12m_spy         526
vol_3m               96
vol_6m              241
drawdown_6m         192
drawdown_12m        479
pct_above_200dma    388
adj_close             0
fwd_excess            0
fwd_return            0
dtype: int64
```

[5]:

```
            mom_1m       mom_3m       mom_6m      mom_12m  mom_12m_ex_1m  \
count  6312.000000  6216.000000  6072.000000  5786.000000    5786.000000
mean      0.002863     0.003779     0.005591     0.008231       0.007929
std       1.006955     1.007301     1.008082     1.008781       1.008683
min      -4.941975    -4.616448    -4.855368    -3.909592      -3.897538
25%      -0.568056    -0.579037    -0.569720    -0.529152      -0.536417
50%      -0.047140    -0.071575    -0.105291    -0.147831      -0.152973
75%       0.522415     0.506885     0.454105     0.346506       0.364365
max       6.164418     6.110228     6.533410     6.526852       6.353619

        rel_3m_spy   rel_6m_spy  rel_12m_spy       vol_3m       vol_6m  \
count  6216.000000  6072.000000  5786.000000  6216.000000  6071.000000
mean      0.003779     0.005591     0.008231     0.010924     0.010749
std       1.007301     1.008082     1.008781     1.006466     1.006531
min      -4.616448    -4.855368    -3.909592    -1.746743    -1.669030
25%      -0.579037    -0.569720    -0.529152    -0.641027    -0.635937
50%      -0.071575    -0.105291    -0.147831    -0.256803    -0.265028
75%       0.506885     0.454105     0.346506     0.366834     0.377619
max       6.110228     6.533410     6.526852     6.000460     5.679121

       drawdown_6m  drawdown_12m  pct_above_200dma    adj_close   fwd_excess  \
count  6120.000000   5833.000000       5924.000000  6312.000000  6312.000000
```

```
mean     -0.002114     -0.001277          0.005295    63.114141     0.034056
std       1.005151      1.005172          1.007927    60.450825     0.233581
min      -6.121311     -6.307731         -5.148334     0.210643    -0.742708
25%      -0.356688     -0.361195         -0.568757    22.894295    -0.083384
50%       0.397425      0.369031         -0.063726    45.524954     0.011648
75%       0.630456      0.639367          0.499260    83.734716     0.111277
max       1.660461      1.562580          5.877712   662.283752     4.020147

        fwd_return
count  6312.000000
mean      0.110603
std       0.249181
min      -0.761706
25%      -0.018780
50%       0.084548
75%       0.200053
max       4.103457
```

[6]:
```python
# Analyze the target variable 'fwd_excess'
print(training_data['fwd_excess'].describe())

# By ticker - mean
ticker_group = training_data.groupby("Ticker")['fwd_excess'].describe().
 ↪sort_values('mean', ascending=False)
ticker_group
```

```
count    6312.000000
mean        0.034056
std         0.233581
min        -0.742708
25%        -0.083384
50%         0.011648
75%         0.111277
max         4.020147
Name: fwd_excess, dtype: float64
```

[6]:

Ticker	count	mean	std	min	25%	50%	75%	\
TSLA	132.0	0.343586	0.800536	-0.511812	-0.111229	0.086031	0.365226	
NFLX	137.0	0.203107	0.501023	-0.698185	-0.106404	0.151981	0.385885	
NVDA	137.0	0.195739	0.367165	-0.436245	-0.029262	0.085357	0.415260	
AMD	137.0	0.147773	0.502850	-0.742708	-0.204725	0.041220	0.410807	
AVGO	137.0	0.105272	0.165957	-0.196993	-0.028021	0.086841	0.197614	
META	109.0	0.101172	0.218840	-0.293504	-0.025067	0.062503	0.184757	
AMZN	137.0	0.095740	0.205740	-0.321286	-0.041970	0.070517	0.245249	
ASML	137.0	0.093080	0.146323	-0.213727	0.021243	0.095200	0.168726	
MU	137.0	0.091248	0.348412	-0.472740	-0.212027	0.052406	0.326857	
AAPL	137.0	0.084553	0.189100	-0.430331	-0.043496	0.103470	0.213669	

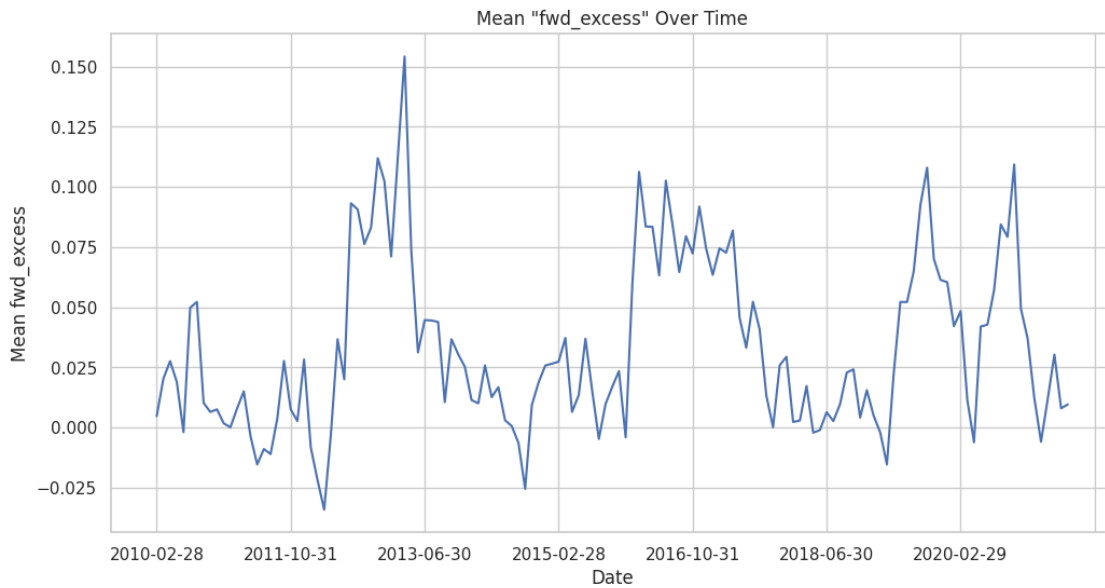
LRCX	136.0	0.082489	0.189970	-0.358800	-0.043723	0.073878	0.212969
UNH	137.0	0.061534	0.104059	-0.199528	-0.002127	0.055271	0.122415
MA	137.0	0.061468	0.123499	-0.227333	-0.018367	0.060947	0.135700
HD	137.0	0.056975	0.106610	-0.181832	-0.029390	0.055351	0.133885
MSFT	137.0	0.055061	0.112606	-0.178858	-0.025678	0.071170	0.129963
V	137.0	0.043824	0.118697	-0.236447	-0.022832	0.060026	0.127745
GOOGL	137.0	0.043584	0.119453	-0.275899	-0.032913	0.031590	0.107088
COST	137.0	0.040447	0.103260	-0.215480	-0.032032	0.049181	0.104214
LLY	137.0	0.039597	0.155044	-0.281866	-0.070174	0.042033	0.142176
NVO	136.0	0.034361	0.168535	-0.455729	-0.072839	0.033267	0.157097
ABBV	101.0	0.019997	0.139895	-0.374199	-0.069675	0.016518	0.087363
MS	137.0	0.017154	0.189100	-0.365527	-0.098558	-0.008541	0.134341
JPM	137.0	0.013140	0.130747	-0.310867	-0.071509	0.000684	0.078843
CAT	137.0	0.009842	0.169436	-0.288168	-0.115459	-0.003376	0.116303
AXP	136.0	0.006312	0.121268	-0.299514	-0.085429	0.020831	0.085120
BAC	137.0	0.004568	0.205949	-0.471089	-0.116828	-0.009026	0.112951
ORCL	137.0	-0.001420	0.103014	-0.238922	-0.074479	0.000980	0.068398
GS	136.0	-0.004437	0.172183	-0.277798	-0.122770	-0.030493	0.064762
BABA	81.0	-0.007257	0.252590	-0.593218	-0.170324	-0.013610	0.165658
SAP	136.0	-0.007582	0.123064	-0.404325	-0.071001	0.003200	0.072925
AZN	137.0	-0.012436	0.134459	-0.343772	-0.100816	-0.012155	0.068342
WMT	137.0	-0.012479	0.115432	-0.271324	-0.096540	-0.018307	0.076524
PM	136.0	-0.012948	0.129421	-0.306727	-0.102532	-0.015953	0.088798
CSCO	137.0	-0.013090	0.147138	-0.393400	-0.115434	0.015670	0.094784
JNJ	137.0	-0.013163	0.093064	-0.213838	-0.089048	-0.013488	0.053065
BRK-B	137.0	-0.013393	0.074213	-0.225964	-0.059383	-0.002022	0.035044
TM	136.0	-0.014292	0.108605	-0.228667	-0.087705	-0.010048	0.057065
MRK	136.0	-0.015412	0.123722	-0.331786	-0.073730	-0.021238	0.051595
PG	137.0	-0.017653	0.109243	-0.302303	-0.082535	-0.027336	0.044782
WFC	137.0	-0.018506	0.176817	-0.585536	-0.098657	-0.010153	0.051747
NVS	136.0	-0.022991	0.104854	-0.265649	-0.106262	-0.015506	0.069911
KO	137.0	-0.023711	0.091391	-0.248644	-0.092211	-0.035513	0.037906
CVX	137.0	-0.030075	0.116919	-0.356030	-0.100413	-0.019923	0.049547
GE	137.0	-0.043557	0.230219	-0.607044	-0.125743	-0.045590	0.064691
XOM	137.0	-0.045409	0.148135	-0.403921	-0.122668	-0.050387	0.013505
IBM	136.0	-0.049794	0.104866	-0.250915	-0.123938	-0.043282	0.019547
HSBC	136.0	-0.062829	0.146881	-0.611188	-0.135322	-0.079996	-0.008727
PLTR	9.0	-0.077458	0.445365	-0.575596	-0.322350	-0.092884	-0.033100

Ticker	max
TSLA	4.020147
NFLX	2.376051
NVDA	1.423459
AMD	2.323218
AVGO	0.566537
META	1.033359

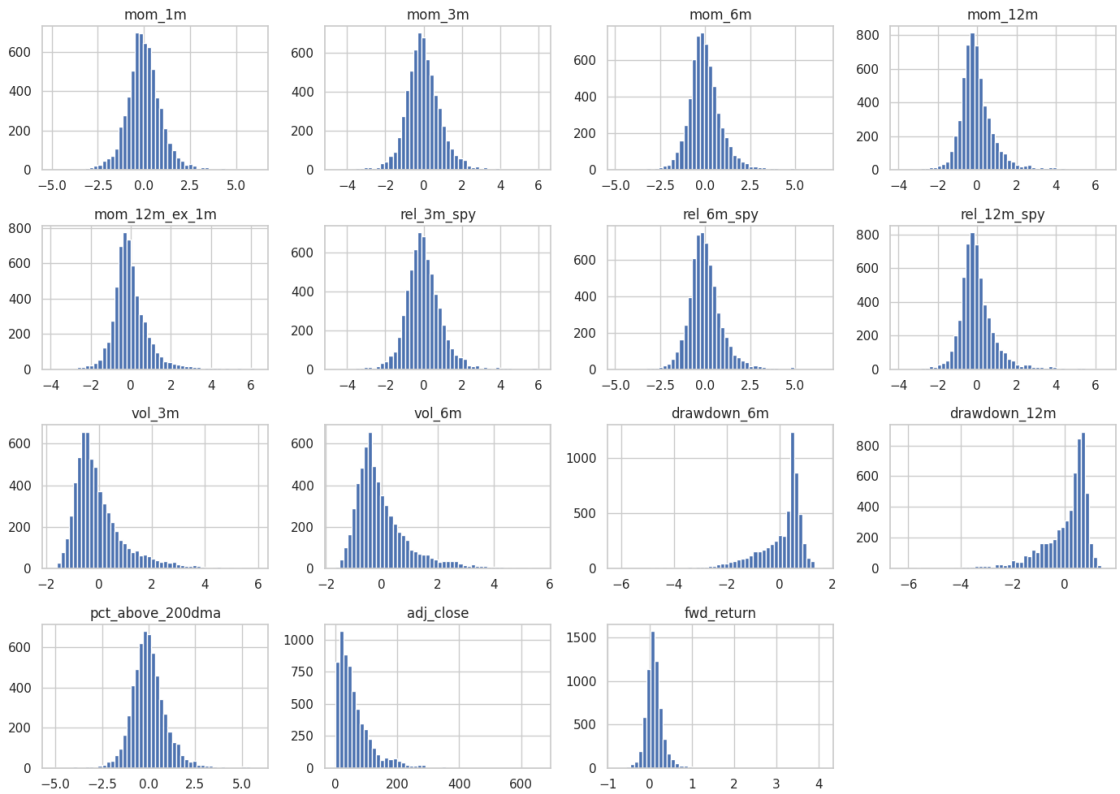
```
AMZN    0.640745
ASML    0.513613
MU      1.122790
AAPL    0.705514
LRCX    0.610807
UNH     0.378020
MA      0.445558
HD      0.340476
MSFT    0.329686
V       0.306520
GOOGL   0.439198
COST    0.339299
LLY     0.439670
NVO     0.389702
ABBV    0.474103
MS      0.577069
JPM     0.415049
CAT     0.436448
AXP     0.404100
BAC     0.600587
ORCL    0.260872
GS      0.572993
BABA    0.612718
SAP     0.294701
AZN     0.453215
WMT     0.267803
PM      0.278477
CSCO    0.304018
JNJ     0.231869
BRK-B   0.163082
TM      0.377056
MRK     0.345451
PG      0.289536
WFC     0.828957
NVS     0.205118
KO      0.191718
CVX     0.305307
GE      0.918638
XOM     0.540011
IBM     0.219798
HSBC    0.332654
PLTR    0.986837
```

[7]:
```
# Average 'fwd_excess' over time
date_group = training_data.groupby('Date')['fwd_excess'].mean()
date_group.plot(title='Mean "fwd_excess" Over Time', figsize=(12,6),␣
 ↪ylabel='Mean fwd_excess', xlabel='Date')
```

[7]: <Axes: title={'center': 'Mean "fwd_excess" Over Time'}, xlabel='Date', ylabel='Mean fwd_excess'>



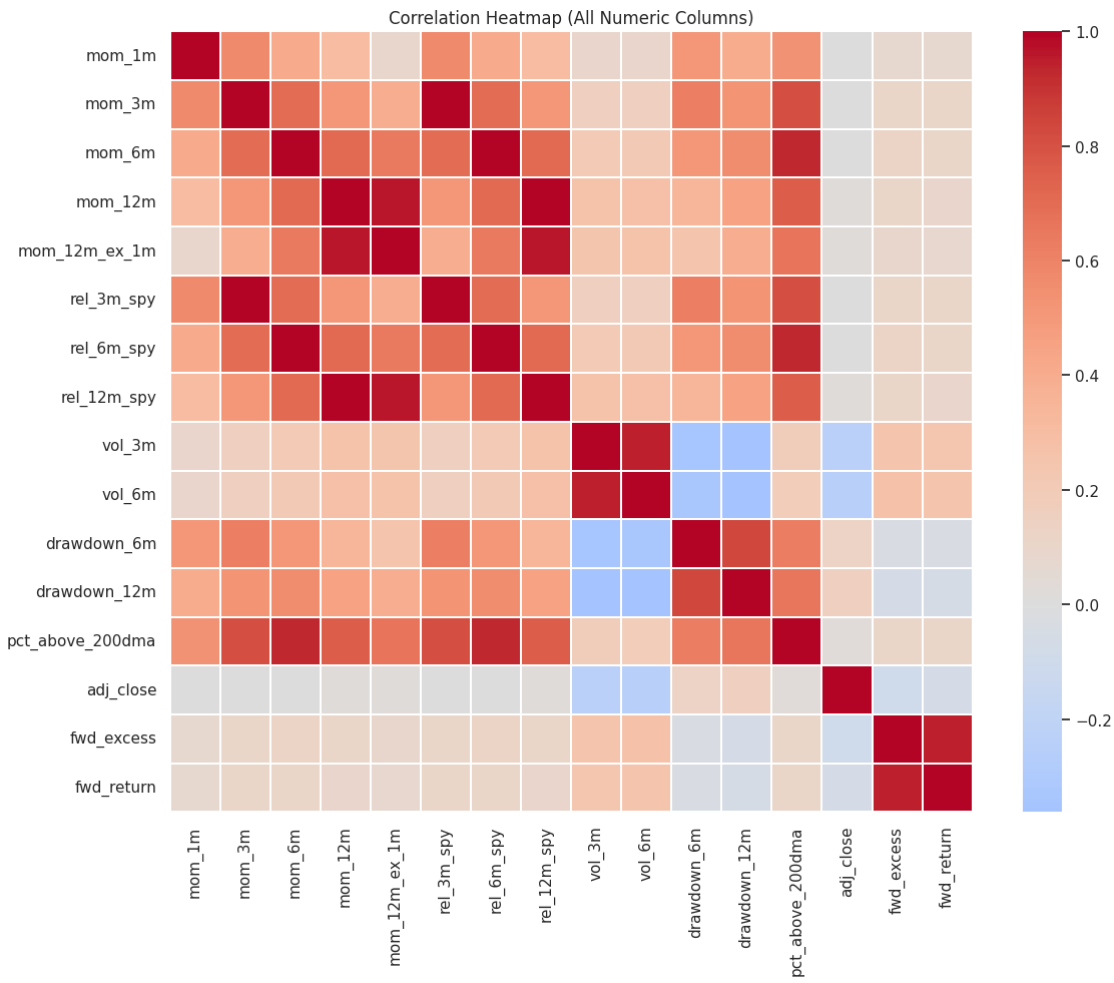
[8]:
```
# Histogram of variables

cols = training_data.columns.drop(['fwd_excess','Date'])
training_data[cols].hist(figsize=(14, 10), bins=50)
plt.tight_layout()
plt.show()
```



[9]:
```
# Correaltion Anlaysis

# select only numeric columns for correlation
numeric_data = training_data.select_dtypes(include=[np.number])
corr = training_data.select_dtypes(include='number').corr()

fwd_excess_corr = numeric_data.corrwith(numeric_data['fwd_excess']).
 ↪sort_values()
print("Correlation of features with 'fwd_excess':")
print(fwd_excess_corr.tail(15))

plt.figure(figsize=(12, 10))
sns.heatmap(corr, cmap='coolwarm', center=0, linewidths=0.2)
plt.title('Correlation Heatmap (All Numeric Columns)')
plt.tight_layout()
plt.show()
```

```
Correlation of features with 'fwd_excess':
drawdown_12m       -0.061281
drawdown_6m        -0.036104
mom_1m              0.078982
mom_12m_ex_1m       0.086621
```

```
rel_12m_spy         0.105410
mom_12m             0.105410
mom_3m              0.106769
rel_3m_spy          0.106769
pct_above_200dma    0.110647
rel_6m_spy          0.117103
mom_6m              0.117103
vol_3m              0.251590
vol_6m              0.275100
fwd_return          0.948606
fwd_excess          1.000000
dtype: float64
```



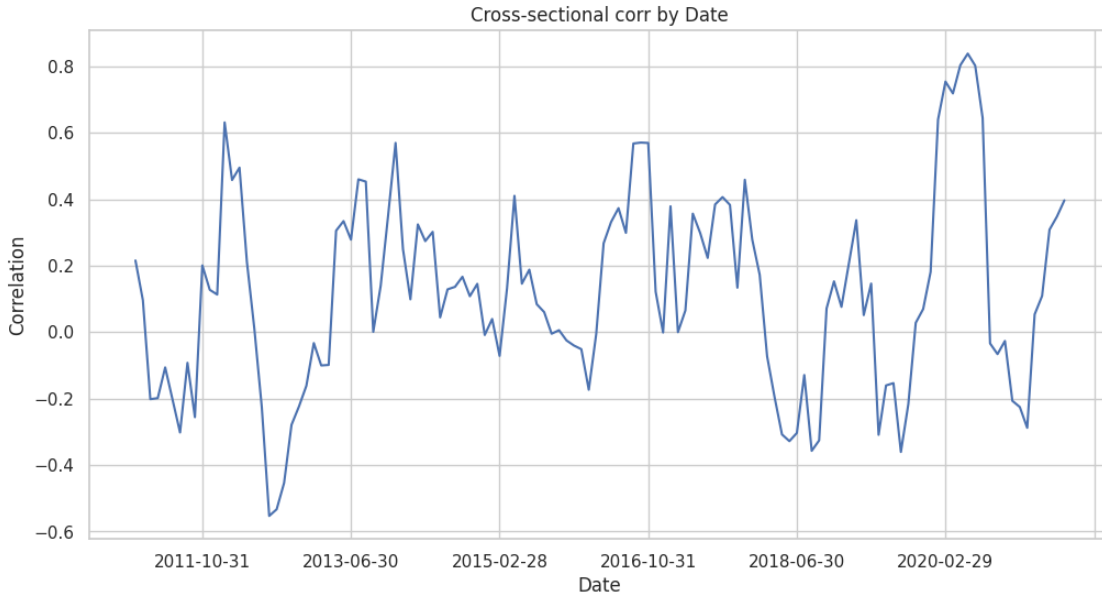
[10]:
```python
# Example rolling correlation by date
tmp = training_data.reset_index()
tmp = tmp.sort_values('Date')
```

```
roll = tmp.groupby('Date')[['mom_12m','fwd_excess']].mean().rolling(12).corr().
 ↪iloc[0::2,1]
roll.plot(title='Rolling corr: mom_12m vs fwd_excess', figsize=(14,8),␣
 ↪ylabel='Correlation', xlabel='Date')
```

[10]: <Axes: title={'center': 'Rolling corr: mom_12m vs fwd_excess'}, xlabel='Date', ylabel='Correlation'>

[11]:
```
'''
Cross - sectional analysis per date
For each month, examine signal effectiveness cross-sectionally
Rank features by how well they explain fwd_excess
'''

# Cross-sectional correlation per date
cs_corr = tmp.groupby('Date').apply(lambda g: g['mom_12m'].
 ↪corr(g['fwd_excess']))
cs_corr.plot(title='Cross-sectional corr by Date',figsize=(12,6),␣
 ↪ylabel='Correlation', xlabel='Date')
```

[11]: <Axes: title={'center': 'Cross-sectional corr by Date'}, xlabel='Date', ylabel='Correlation'>



[12]:
```
'''
Potential Outlier influence
Extreme fwd_excess tails: which tickers/dates drive them?
Winsorization sensitivity checks
'''

q = training_data['fwd_excess'].quantile([0.01, 0.99])
tail = training_data[(training_data['fwd_excess']<=q.iloc[0]) |␣
↪(training_data['fwd_excess']>=q.iloc[1])]
tail.reset_index().groupby('Ticker').size().sort_values(ascending=False).
↪head(10)
```

[12]:
```
Ticker
AMD     22
TSLA    21
NFLX    20
NVDA    13
GE      11
MU       9
BABA     7
WFC      6
BAC      4
PLTR     3
dtype: int64
```

[13]:
```
zip_path = "HelpHerInvest/Data/stock_symbols_new.csv.zip"
```

```python
with zipfile.ZipFile(zip_path) as z:
    df = pd.read_csv(z.open("stock_symbols_new.csv"))

df.head()
```

[13]:

```
  symbol    company_name                   address1           city state  \
0   NVDA     NVIDIA CORP  2788 San Tomas Expressway    Santa Clara    CA
1  GOOGL   Alphabet Inc.  1600 Amphitheatre Parkway  Mountain View    CA
2   AAPL      Apple Inc.         One Apple Park Way      Cupertino    CA
3   MSFT  MICROSOFT CORP          One Microsoft Way        Redmond    WA
4   AMZN  AMAZON COM INC     410 Terry Avenue North        Seattle    WA

          zip        country           phone                    website  \
0       95051  United States    408 486 2000     https://www.nvidia.com
1       94043  United States    650-253-0000            https://abc.xyz
2       95014  United States  (408) 996-1010      https://www.apple.com
3  98052-6399  United States    425 882 8080  https://www.microsoft.com
4  98109-5210  United States    206 266 1000     https://www.amazon.com

                        industry                   industryKey  \
0                  Semiconductors                semiconductors
1  Internet Content & Information  internet-content-information
2            Consumer Electronics          consumer-electronics
3       Software - Infrastructure       software-infrastructure
4                 Internet Retail               internet-retail

                    industryDisp                  sector  \
0                  Semiconductors              Technology
1  Internet Content & Information  Communication Services
2            Consumer Electronics              Technology
3       Software - Infrastructure              Technology
4                 Internet Retail       Consumer Cyclical

                sectorKey              sectorDisp  \
0              technology              Technology
1  communication-services  Communication Services
2              technology              Technology
3              technology              Technology
4       consumer-cyclical       Consumer Cyclical

   longBusinessSummary  \
0  NVIDIA Corporation, a computing infrastructure company, provides graphics and
compute and networking solutions in the United States, Singapore, Taiwan, China,
Hong Kong, and in…
1  Alphabet Inc. offers various products and platforms in the United States,
Europe, the Middle East, Africa, the Asia-Pacific, Canada, and Latin America. It
operates through Goog…
```

```
2  Apple Inc. designs, manufactures, and markets smartphones, personal
computers, tablets, wearables, and accessories worldwide. The company offers
iPhone, a line of smartphones; …
3  Microsoft Corporation develops and supports software, services, devices, and
solutions worldwide. The company's Productivity and Business Processes segment
offers Microsoft 365…
4  Amazon.com, Inc. engages in the retail sale of consumer products,
advertising, and subscriptions service through online and physical stores in
North America and internationally…

   fullTimeEmployees  \
0            36000.0
1           190167.0
2           166000.0
3           228000.0
4          1578000.0

       companyOfficers  \
0  [{'maxAge': 1, 'name': 'Mr. Jen-Hsun  Huang', 'age': 62, 'title': 'Co-
Founder, CEO, President & Director', 'yearBorn': 1963, 'fiscalYear': 2025,
'totalPay': 11054945, 'exercise…
1  [{'maxAge': 1, 'name': 'Mr. Sundar  Pichai', 'age': 52, 'title': 'CEO &
Director', 'yearBorn': 1973, 'fiscalYear': 2024, 'totalPay': 10319413,
'exercisedValue': 0, 'unexercised…
2  [{'maxAge': 1, 'name': 'Mr. Timothy D. Cook', 'age': 64, 'title': 'CEO &
Director', 'yearBorn': 1961, 'fiscalYear': 2025, 'totalPay': 16759518,
'exercisedValue': 0, 'unexercise…
3  [{'maxAge': 1, 'name': 'Mr. Satya  Nadella', 'age': 58, 'title': 'Chairman &
CEO', 'yearBorn': 1967, 'fiscalYear': 2025, 'totalPay': 12251294,
'exercisedValue': 0, 'unexercised…
4  [{'maxAge': 1, 'name': 'Mr. Jeffrey P. Bezos', 'age': 61, 'title': 'Founder &
Executive Chairman', 'yearBorn': 1964, 'fiscalYear': 2024, 'totalPay': 1681840,
'exercisedValue': …

   auditRisk  boardRisk  compensationRisk  shareHolderRightsRisk  overallRisk  \
0        5.0       10.0               4.0                    8.0          8.0
1        7.0        9.0              10.0                   10.0         10.0
2        7.0        1.0               3.0                    1.0          1.0
3        9.0        8.0               5.0                    2.0          5.0
4        2.0        8.0              10.0                    3.0          9.0

   governanceEpochDate  compensationAsOfEpochDate  \
0         1.768176e+09               1.767139e+09
1         1.768176e+09               1.735603e+09
2         1.768176e+09               1.767139e+09
3         1.768176e+09               1.767139e+09
4         1.768176e+09               1.735603e+09
```

```
                                                          irWebsite  \
0  http://phx.corporate-ir.net/phoenix.zhtml?c=116466&p=irol-IRHome
1                                                                NaN
2                                          http://investor.apple.com/
3                     http://www.microsoft.com/investor/default.aspx
4   http://phx.corporate-ir.net/phoenix.zhtml?c=97664&p=irol-irhome

  executiveTeam   maxAge  priceHint  previousClose     open    dayLow  \
0            []  86400.0        2.0         178.07  179.050  178.4000
1            []  86400.0        2.0         322.00  320.865  319.3518
2            []  86400.0        2.0         246.70  248.710  245.1900
3            []  86400.0        2.0         454.52  452.350  438.6850
4            []  86400.0        2.0         231.00  231.085  226.8800

    dayHigh  regularMarketPreviousClose  regularMarketOpen  \
0  185.3790                      178.07            179.050
1  332.4799                      322.00            320.865
2  251.5600                      246.70            248.710
3  452.6900                      454.52            452.350
4  232.3000                      231.00            231.085

   regularMarketDayLow  regularMarketDayHigh  dividendRate  dividendYield  \
0             178.4000              185.3790          0.04           0.02
1             319.3518              332.4799          0.84           0.26
2             245.1900              251.5600          1.04           0.42
3             438.6850              452.6900          3.64           0.80
4             226.8800              232.3000           NaN            NaN

   exDividendDate  payoutRatio  fiveYearAvgDividendYield   beta  trailingPE  \
0    1.764806e+09       0.0099                      0.06  2.314   45.376240
1    1.765152e+09       0.0809                       NaN  1.086   32.384613
2    1.762733e+09       0.1367                      0.52  1.093   33.241610
3    1.771459e+09       0.2361                      0.80  1.073   31.631765
4             NaN       0.0000                       NaN  1.376   32.717113

   forwardPE       volume  regularMarketVolume  averageVolume  \
0  23.920063  198539922.0          198539922.0    186131977.0
1  29.139809   35014518.0           35014518.0     36498273.0
2  27.102331   54151253.0           54151253.0     46047281.0
3  23.700440   37424042.0           37424042.0     24331393.0
4  29.431673   46594728.0           46594728.0     44821501.0

   averageVolume10days  averageDailyVolume10Day     bid     ask  bidSize  \
0          170859330.0              170859330.0  182.93  183.39     20.0
1           32441880.0               32441880.0  328.10  328.45      3.0
2           51370470.0               51370470.0  247.43  261.14      4.0
```

```
3          24906140.0               24906140.0  443.81  444.42      1.0
4          42215840.0               42215840.0  231.15  231.40      4.0

   askSize            marketCap  fiftyTwoWeekLow  fiftyTwoWeekHigh  \
0      1.0   4,463,292,579,840            86.62            212.19
1      1.0   3,977,378,529,280           140.53            340.49
2      1.0   3,659,363,778,560           169.21            288.62
3      1.0   3,301,143,478,272           344.79            555.45
4      7.0   2,472,753,758,208           161.38            258.60

   allTimeHigh  allTimeLow  priceToSalesTrailing12Months  fiftyDayAverage  \
0       212.19    0.033333                     23.849764         184.2150
1       340.49    2.401401                     10.318096         310.1148
2       288.62    0.049107                      8.793144         271.0410
3       555.45    0.088542                     11.235563         483.7818
4       258.60    0.065625                      3.576806         232.9836

   twoHundredDayAverage  trailingAnnualDividendRate  \
0             164.39460                        0.04
1             227.18805                        1.02
2             234.16930                        1.02
3             482.62060                        3.40
4             219.39810                        0.00

   trailingAnnualDividendYield currency tradeable  enterpriseValue  \
0                     0.000225      USD     False     4.277320e+12
1                     0.003168      USD     False     3.831270e+12
2                     0.004135      USD     False     3.703010e+12
3                     0.007480      USD     False     3.396530e+12
4                     0.000000      USD     False     2.535680e+12

   profitMargins   floatShares  sharesOutstanding  sharesShort  \
0        0.53007  2.333067e+10       2.430500e+10  256762405.0
1        0.32233  1.079840e+10       5.818000e+09   82670176.0
2        0.26915  1.475035e+10       1.469793e+10  112732788.0
3        0.35707  7.421452e+09       7.432378e+09   52944379.0
4        0.11063  9.701906e+09       1.069022e+10   73419039.0

   sharesShortPriorMonth  sharesShortPreviousMonthDate  dateShortInterest  \
0            232898275.0                  1.764288e+09       1.767139e+09
1             79334780.0                  1.764288e+09       1.767139e+09
2            129458559.0                  1.764288e+09       1.767139e+09
3             61948927.0                  1.764288e+09       1.767139e+09
4             73266891.0                  1.764288e+09       1.767139e+09

   sharesPercentSharesOut  heldPercentInsiders  heldPercentInstitutions  \
0                  0.0106              0.04329                  0.69491
```

1	0.0069	0.00568	0.81179
2	0.0076	0.01702	0.64820
3	0.0071	0.00066	0.75803
4	0.0069	0.09178	0.66816

	shortRatio	shortPercentOfFloat	impliedSharesOutstanding	bookValue	\
0	1.56	0.0110	2.434700e+10	4.892	
1	2.68	0.0142	1.211212e+10	32.033	
2	2.68	0.0076	1.477635e+10	4.991	
3	2.35	0.0071	7.433166e+09	48.840	
4	2.05	0.0076	1.069022e+10	34.587	

	priceToBook	lastFiscalYearEnd	nextFiscalYearEnd	mostRecentQuarter	\
0	37.473427	1.737850e+09	1.769386e+09	1.761437e+09	
1	10.251303	1.735603e+09	1.767139e+09	1.759190e+09	
2	49.619312	1.758931e+09	1.790467e+09	1.758931e+09	
3	9.093161	1.751242e+09	1.782778e+09	1.759190e+09	
4	6.687772	1.735603e+09	1.767139e+09	1.759190e+09	

	earningsQuarterlyGrowth	netIncomeToCommon	trailingEps	forwardEps	\
0	0.653	9.919800e+10	4.04	7.66386	
1	0.330	1.242510e+11	10.14	11.26912	
2	0.864	1.120100e+11	7.45	9.13759	
3	0.125	1.049120e+11	14.04	18.73847	
4	0.382	7.648200e+10	7.07	7.85922	

	lastSplitFactor	lastSplitDate	enterpriseToRevenue	enterpriseToEbitda	\
0	10:01	1.717978e+09	22.856	37.954	
1	20:01	1.658102e+09	9.939	26.391	
2	4:01	1.598832e+09	8.898	25.582	
3	2:01	1.045526e+09	11.560	20.407	
4	20:01	1.654474e+09	3.668	18.151	

	52WeekChange	SandP52WeekChange	lastDividendValue	lastDividendDate	\
0	0.210784	0.116735	0.01	1.764806e+09	
1	0.623229	0.116735	0.21	1.765152e+09	
2	0.102176	0.116735	0.26	1.762733e+09	
3	0.018646	0.116735	0.91	1.763597e+09	
4	-0.017063	0.116735	NaN	NaN	

	quoteType	currentPrice	targetHighPrice	…	operatingMargins	\
0	EQUITY	183.32	352.0	…	0.63169	
1	EQUITY	328.38	432.0	…	0.30512	
2	EQUITY	247.65	350.0	…	0.31647	
3	EQUITY	444.11	730.0	…	0.48873	
4	EQUITY	231.31	360.0	…	0.11057	

```
   financialCurrency  symbol.1  language region  typeDisp  \
0               USD      NVDA     en-US     US    Equity
1               USD     GOOGL     en-US     US    Equity
2               USD      AAPL     en-US     US    Equity
3               USD      MSFT     en-US     US    Equity
4               USD      AMZN     en-US     US    Equity

          quoteSourceName  triggerable  customPriceAlertConfidence  \
0  Nasdaq Real Time Price         True                        HIGH
1  Nasdaq Real Time Price         True                        HIGH
2  Nasdaq Real Time Price         True                        HIGH
3  Nasdaq Real Time Price         True                        HIGH
4  Nasdaq Real Time Price         True                        HIGH

   marketState               longName  sourceInterval  exchangeDataDelayedBy  \
0         POST     NVIDIA Corporation            15.0                    0.0
1         POST          Alphabet Inc.            15.0                    0.0
2         POST             Apple Inc.            15.0                    0.0
3         POST  Microsoft Corporation            15.0                    0.0
4         POST       Amazon.com, Inc.            15.0                    0.0

          prevName  nameChangeDate  averageAnalystRating  cryptoTradeable  \
0  Usual Stablecoin         1/21/26      1.3 - Strong Buy            False
1              NaN             NaN      1.4 - Strong Buy            False
2              NaN             NaN            2.0 - Buy            False
3              NaN             NaN      1.3 - Strong Buy            False
4              NaN             NaN      1.3 - Strong Buy            False

              shortName  corporateActions  postMarketTime  regularMarketTime  \
0     NVIDIA Corporation                []    1.769039e+09       1.769029e+09
1          Alphabet Inc.                []    1.769039e+09       1.769029e+09
2            Apple Inc.                 []    1.769039e+09       1.769029e+09
3  Microsoft Corporation                []    1.769039e+09       1.769029e+09
4       Amazon.com, Inc.                []    1.769039e+09       1.769029e+09

   exchange  messageBoardId  exchangeTimezoneName  exchangeTimezoneShortName  \
0       NMS     finmb_32307      America/New_York                        EST
1       NMS     finmb_29096      America/New_York                        EST
2       NMS     finmb_24937      America/New_York                        EST
3       NMS     finmb_21835      America/New_York                        EST
4       NMS     finmb_18749      America/New_York                        EST

  gmtOffSetMilliseconds     market esgPopulated hasPrePostMarketData  \
0           -18000000.0  us_market        False                 True
1           -18000000.0  us_market        False                 True
2           -18000000.0  us_market        False                 True
3           -18000000.0  us_market        False                 True
```

```
4           -18000000.0  us_market        False                 True

  firstTradeDateMilliseconds postMarketChangePercent postMarketPrice  \
0               9.170150e+11                0.139095        183.5750
1               1.092920e+12                0.240578        329.1700
2               3.454790e+11                0.327080        248.4600
3               5.111080e+11                0.425553        445.9999
4               8.637030e+11                0.237777        231.8600

  postMarketChange regularMarketChange regularMarketDayRange  \
0         0.254990            5.250000       178.4 - 185.379
1         0.790009            6.380005   319.3518 - 332.4799
2         0.810013            0.949997       245.19 - 251.56
3         1.889923          -10.410004      438.685 - 452.69
4         0.550003            0.309998        226.88 - 232.3

   fullExchangeName  averageDailyVolume3Month fiftyTwoWeekLowChange  \
0          NasdaqGS               186131977.0             96.700005
1          NasdaqGS                36498273.0            187.850000
2          NasdaqGS                46047281.0             78.439990
3          NasdaqGS                24331393.0             99.319980
4          NasdaqGS                44821501.0             69.929990

  fiftyTwoWeekLowChangePercent fiftyTwoWeekRange fiftyTwoWeekHighChange  \
0                     1.116370    86.62 - 212.19             -28.869995
1                     1.336725   140.53 - 340.49             -12.109985
2                     0.463566   169.21 - 288.62             -40.970000
3                     0.288059   344.79 - 555.45            -111.340030
4                     0.433325    161.38 - 258.6             -27.290009

  fiftyTwoWeekHighChangePercent fiftyTwoWeekChangePercent  dividendDate  \
0                     -0.136057                 21.078396  1.766707e+09
1                     -0.035566                 62.322926  1.765757e+09
2                     -0.141951                 10.217571  1.762992e+09
3                     -0.200450                  1.864636  1.773274e+09
4                     -0.105530                 -1.706308           NaN

   earningsTimestamp earningsTimestampStart earningsTimestampEnd  \
0       1.772053e+09           1.772053e+09         1.772053e+09
1       1.770239e+09           1.770239e+09         1.770239e+09
2       1.769720e+09           1.769720e+09         1.769720e+09
3       1.769634e+09           1.769634e+09         1.769634e+09
4       1.761854e+09           1.770325e+09         1.770325e+09

  earningsCallTimestampStart earningsCallTimestampEnd  isEarningsDateEstimate  \
0               1.763590e+09             1.763590e+09                   False
1               1.770241e+09             1.770241e+09                   False
```

```
2              1.769724e+09            1.769724e+09                  False
3              1.769639e+09            1.769639e+09                  False
4              1.761858e+09            1.761858e+09                   True

  epsTrailingTwelveMonths epsForward epsCurrentYear  priceEpsCurrentYear  \
0                    4.04    7.66386        4.69210            39.069927
1                   10.14   11.26912       10.54084            31.153114
2                    7.45    9.13759        8.24255            30.045313
3                   14.04   18.73847       16.44100            27.012346
4                    7.07    7.85922        7.07210            32.707397

   fiftyDayAverageChange  fiftyDayAverageChangePercent  \
0              -0.894989                     -0.004858
1              18.265198                      0.058898
2             -23.390991                     -0.086301
3             -39.671814                     -0.082004
4              -1.673599                     -0.007183

   twoHundredDayAverageChange  twoHundredDayAverageChangePercent  \
0                   18.925400                           0.115122
1                  101.191956                           0.445411
2                   13.480698                           0.057568
3                  -38.510620                          -0.079795
4                   11.911896                           0.054294

  regularMarketChangePercent regularMarketPrice  displayName  \
0                   2.948279             183.32       NVIDIA
1                   1.981368             328.38          NaN
2                   0.385082             247.65        Apple
3                  -2.290329             444.11    Microsoft
4                   0.134198             231.31   Amazon.com

   trailingPegRatio  ipoExpectedDate yield  totalAssets  navPrice  category  \
0            0.6722              NaN   NaN          NaN       NaN       NaN
1            1.7001              NaN   NaN          NaN       NaN       NaN
2            2.4965              NaN   NaN          NaN       NaN       NaN
3            1.8385              NaN   NaN          NaN       NaN       NaN
4            1.5853              NaN   NaN          NaN       NaN       NaN

   ytdRETURN  beta3Year  fundFamily  fundInceptionDate  legalType  \
0        NaN        NaN         NaN                NaN        NaN
1        NaN        NaN         NaN                NaN        NaN
2        NaN        NaN         NaN                NaN        NaN
3        NaN        NaN         NaN                NaN        NaN
4        NaN        NaN         NaN                NaN        NaN

   threeYearAverageReturn fiveYearAverageReturn  trailingThreeMonthReturns  \
```

```
0                     NaN                   NaN                        NaN
1                     NaN                   NaN                        NaN
2                     NaN                   NaN                        NaN
3                     NaN                   NaN                        NaN
4                     NaN                   NaN                        NaN

   trailingThreeMonthNavReturns  netAssets  netExpenseRatio  address2  fax  \
0                           NaN        NaN              NaN       NaN  NaN
1                           NaN        NaN              NaN       NaN  NaN
2                           NaN        NaN              NaN       NaN  NaN
3                           NaN        NaN              NaN       NaN  NaN
4                           NaN        NaN              NaN       NaN  NaN

   industrySymbol  underlyingSymbol  firstTradeDateEpochUtc  timeZoneFullName  \
0             NaN               NaN                     NaN               NaN
1             NaN               NaN                     NaN               NaN
2             NaN               NaN                     NaN               NaN
3             NaN               NaN                     NaN               NaN
4             NaN               NaN                     NaN               NaN

  timeZoneShortName  uuid newSymbol  openInterest  prevExchange  \
0               NaN   NaN       NaN           NaN           NaN
1               NaN   NaN       NaN           NaN           NaN
2               NaN   NaN       NaN           NaN           NaN
3               NaN   NaN       NaN           NaN           NaN
4               NaN   NaN       NaN           NaN           NaN

   exchangeTransferDate address3  morningStarOverallRating  \
0                   NaN      NaN                       NaN
1                   NaN      NaN                       NaN
2                   NaN      NaN                       NaN
3                   NaN      NaN                       NaN
4                   NaN      NaN                       NaN

   morningStarRiskRating annualReportExpenseRatio  lastCapGain  \
0                    NaN                      NaN          NaN
1                    NaN                      NaN          NaN
2                    NaN                      NaN          NaN
3                    NaN                      NaN          NaN
4                    NaN                      NaN          NaN

  annualHoldingsTurnover  prevTicker  tickerChangeDate  newListingDate  \
0                    NaN         NaN               NaN             NaN
1                    NaN         NaN               NaN             NaN
2                    NaN         NaN               NaN             NaN
3                    NaN         NaN               NaN             NaN
4                    NaN         NaN               NaN             NaN
```

```
   delistingDate
0            NaN
1            NaN
2            NaN
3            NaN
4            NaN

[5 rows x 224 columns]
```

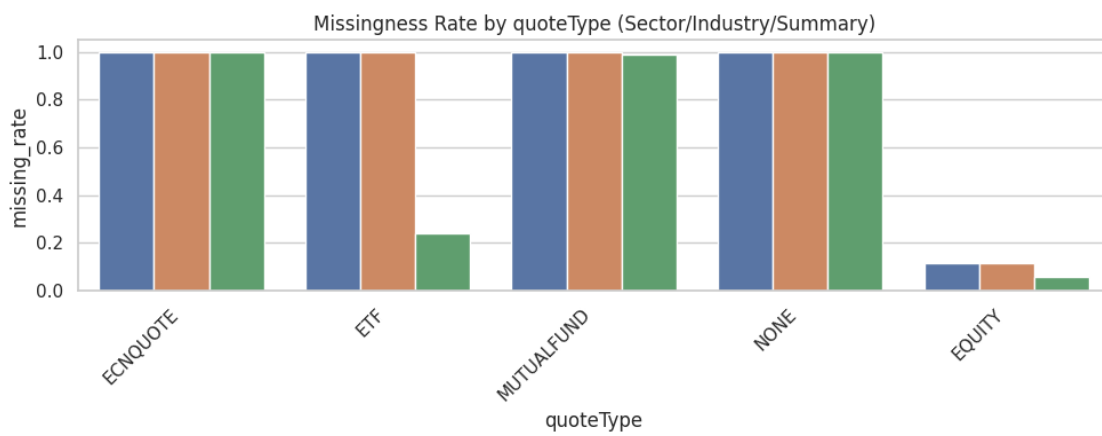
[14]:
```python
# Detect key NLP columns robustly

def find_col(candidates, columns):
    colmap = {c.lower(): c for c in columns}
    for cand in candidates:
        if cand.lower() in colmap:
            return colmap[cand.lower()]
    return None

sector_col = find_col(["sector", "sectorDisp", "sectorKey"], df.columns)
industry_col = find_col(["industry", "industryDisp", "industryKey"], df.columns)
summary_col = find_col(["longBusinessSummary", "businessSummary", "summary",␣
 ↪"description"], df.columns)

symbol_col = find_col(["symbol", "ticker"], df.columns)
name_col = find_col(["company_name", "shortName", "longName", "name"], df.
 ↪columns)
quote_type_col = find_col(["quoteType"], df.columns)

print("Detected columns:")
print("  symbol:", symbol_col)
print("  company:", name_col)
print("  quoteType:", quote_type_col)
print("  sector:", sector_col)
print("  industry:", industry_col)
print("  summary:", summary_col)

assert sector_col and industry_col and summary_col, "Missing required NLP␣
 ↪columns."
```

```
Detected columns:
  symbol: symbol
  company: company_name
  quoteType: quoteType
  sector: sector
  industry: industry
  summary: longBusinessSummary
```

[15]:
```
#  Coverage / missingness (overall + by quoteType)

def missing_report(s: pd.Series, name: str):
    s2 = s.copy()
    null_count = int(s2.isna().sum())
    empty_count = int((s2.fillna("").astype(str).str.strip() == "").sum())
    n = len(s2)
    return {
        "field": name,
        "rows": n,
        "null_count": null_count,
        "empty_or_whitespace_count": empty_count,
        "nonempty_count": n - empty_count,
        "pct_null": null_count / n,
        "pct_empty_or_whitespace": empty_count / n,
    }

coverage = pd.DataFrame([
    missing_report(df[sector_col], sector_col),
    missing_report(df[industry_col], industry_col),
    missing_report(df[summary_col], summary_col),
])

display(coverage)


if quote_type_col:
    def is_missing_text(series):
        return series.isna() | (series.astype(str).str.strip() == "")

    tmp = df.copy()
    tmp["_miss_sector"] = is_missing_text(tmp[sector_col])
    tmp["_miss_industry"] = is_missing_text(tmp[industry_col])
    tmp["_miss_summary"] = is_missing_text(tmp[summary_col])

    by_qt = (tmp.
 ↪groupby(quote_type_col)[["_miss_sector","_miss_industry","_miss_summary"]]
             .mean()
             .sort_values("_miss_sector", ascending=False)
             .reset_index())

    display(by_qt)


    plt.figure(figsize=(10,4))
    melted = by_qt.melt(id_vars=quote_type_col, var_name="field",␣
 ↪value_name="missing_rate")
```

```
    sns.barplot(data=melted, x=quote_type_col, y="missing_rate", hue="field")
    plt.xticks(rotation=45, ha="right")
    plt.title("Missingness Rate by quoteType (Sector/Industry/Summary)")
    plt.tight_layout()
    plt.show()
```

```
                 field   rows  null_count  empty_or_whitespace_count  \
0               sector  10284        1812                       1812
1             industry  10284        1812                       1812
2  longBusinessSummary  10284        1103                       1103

   nonempty_count  pct_null  pct_empty_or_whitespace
0            8472  0.176196                 0.176196
1            8472  0.176196                 0.176196
2            9181  0.107254                 0.107254

    quoteType  _miss_sector  _miss_industry  _miss_summary
0    ECNQUOTE      1.000000        1.000000       1.000000
1         ETF      1.000000        1.000000       0.241206
2  MUTUALFUND      1.000000        1.000000       0.990566
3        NONE      1.000000        1.000000       1.000000
4      EQUITY      0.114178        0.114178       0.055939
```



[16]:
```
# Sector/industry distributions + cross-checks

sector_counts = df[sector_col].value_counts(dropna=False).reset_index()
sector_counts.columns = ["sector", "count"]
industry_counts = df[industry_col].value_counts(dropna=False).reset_index()
industry_counts.columns = ["industry", "count"]

print("Unique sectors:", df[sector_col].nunique(dropna=True))
print("Unique industries:", df[industry_col].nunique(dropna=True))
```

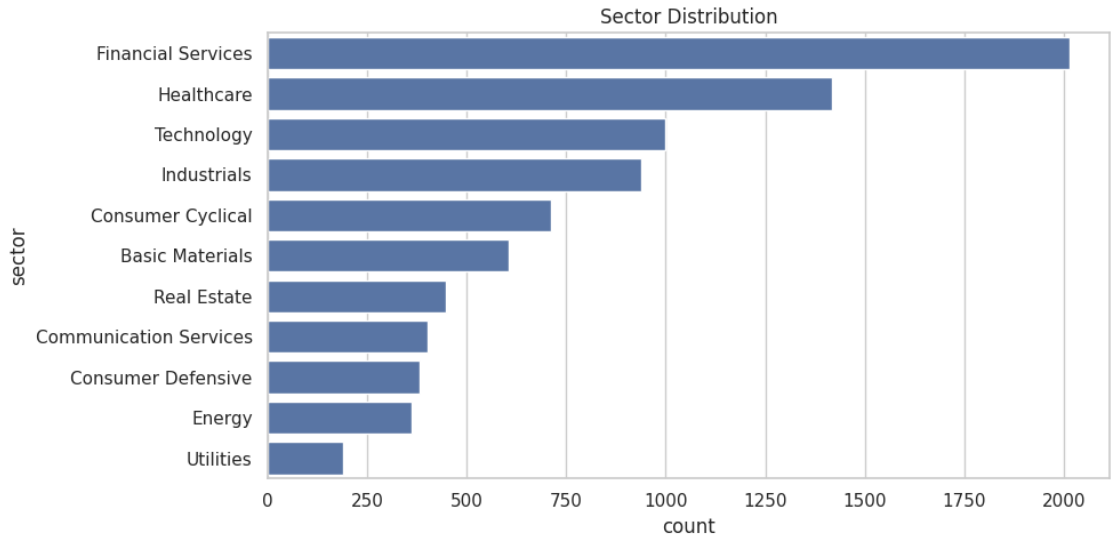
```
display(sector_counts.head(20))
display(industry_counts.head(20))

# Plot sector distribution
plt.figure(figsize=(10,5))
top_sector = sector_counts.dropna().head(20)
sns.barplot(data=top_sector, x="count", y="sector")
plt.title("Sector Distribution")
plt.tight_layout()
plt.show()

# Industries appearing in multiple sectors (potential label issues)
inds_multi_sector = (df.dropna(subset=[sector_col, industry_col])
                       .groupby(industry_col)[sector_col]
                       .nunique()
                       .sort_values(ascending=False)
                       .reset_index()
                       .rename(columns={sector_col:"n_sectors_for_industry"}))

display(inds_multi_sector.head(30))
```

```
Unique sectors: 11
Unique industries: 145
```

	sector	count
0	Financial Services	2014
1	NaN	1812
2	Healthcare	1418
3	Technology	1000
4	Industrials	939
5	Consumer Cyclical	711
6	Basic Materials	607
7	Real Estate	448
8	Communication Services	401
9	Consumer Defensive	381
10	Energy	362
11	Utilities	191

	industry	count
0	NaN	1812
1	Biotechnology	721
2	Asset Management	569
3	Banks - Regional	484
4	Shell Companies	450
5	Software - Application	320
6	Software - Infrastructure	197
7	Medical Devices	182
8	Other Industrial Metals & Mining	175

9	Drug Manufacturers - Specialty & Generic	163
10	Gold	128
11	Capital Markets	121
12	Oil & Gas E&P	120
13	Packaged Foods	108
14	Aerospace & Defense	104
15	Telecom Services	104
16	Specialty Industrial Machinery	102
17	Internet Content & Information	99
18	REIT - Mortgage	99
19	Oil & Gas Midstream	93



	industry	n_sectors_for_industry
0	Advertising Agencies	1
1	Aerospace & Defense	1
2	Agricultural Inputs	1
3	Airlines	1
4	Airports & Air Services	1
5	Aluminum	1
6	Apparel Manufacturing	1
7	Apparel Retail	1
8	Asset Management	1
9	Auto & Truck Dealerships	1
10	Auto Manufacturers	1
11	Auto Parts	1
12	Banks - Diversified	1
13	Banks - Regional	1
14	Beverages - Brewers	1
15	Beverages - Non-Alcoholic	1

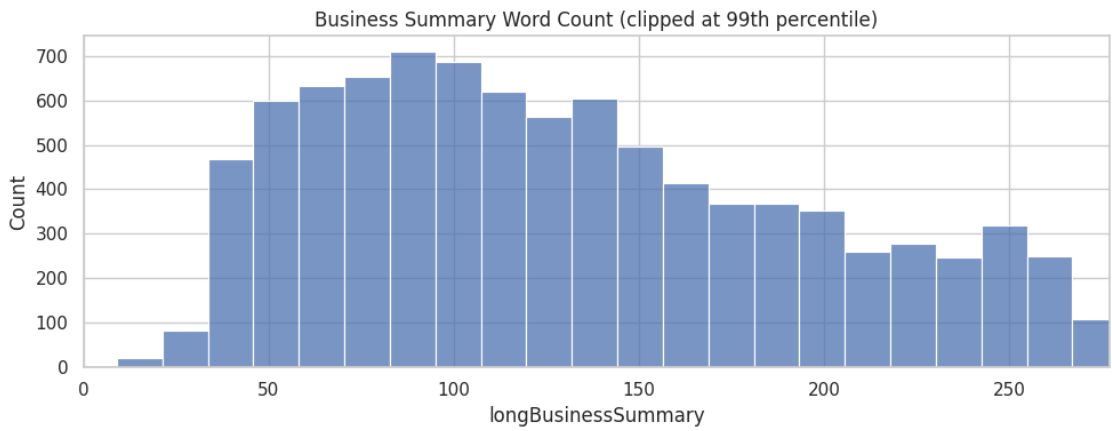
```
16  Beverages - Wineries & Distilleries                        1
17                        Biotechnology                        1
18                         Broadcasting                        1
19                   Building Materials                        1
20        Building Products & Equipment                        1
21        Business Equipment & Supplies                        1
22                      Capital Markets                        1
23                            Chemicals                        1
24                           Coking Coal                       1
25              Communication Equipment                        1
26                    Computer Hardware                        1
27                        Confectioners                        1
28                        Conglomerates                        1
29                  Consulting Services                        1
```

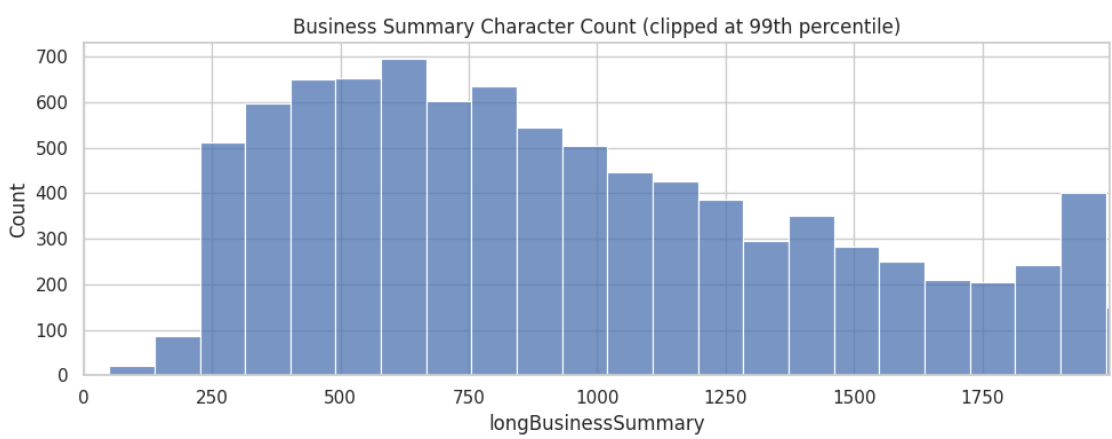
[17]:

```
# Business summary length profiling (plots + outliers)

summ = df[summary_col].fillna("").astype(str).str.strip()
nonempty = summ[summ != ""]

char_len = nonempty.str.len()
word_len = nonempty.str.split().str.len()
sent_len = nonempty.str.count(r"[.!?]") + 1

length_stats = pd.DataFrame({
    "metric": ["char_len","word_len","sent_len"],
    "count": [len(char_len), len(word_len), len(sent_len)],
    "mean": [char_len.mean(), word_len.mean(), sent_len.mean()],
    "median": [char_len.median(), word_len.median(), sent_len.median()],
    "p95": [char_len.quantile(0.95), word_len.quantile(0.95), sent_len.
 ↪quantile(0.95)],
    "p99": [char_len.quantile(0.99), word_len.quantile(0.99), sent_len.
 ↪quantile(0.99)],
    "max": [char_len.max(), word_len.max(), sent_len.max()],
})

display(length_stats)

# Histograms
plt.figure(figsize=(10,4))
sns.histplot(word_len, bins=60)
plt.xlim(0, np.percentile(word_len, 99))
plt.title("Business Summary Word Count (clipped at 99th percentile)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10,4))
```

```
sns.histplot(char_len, bins=60)
plt.xlim(0, np.percentile(char_len, 99))
plt.title("Business Summary Character Count (clipped at 99th percentile)")
plt.tight_layout()
plt.show()

# Very short summaries for inspection
very_short_idx = nonempty.index[word_len < 15]
short_examples = df.loc[very_short_idx, [c for c in [symbol_col, name_col,␣
↪quote_type_col, sector_col, industry_col, summary_col] if c]].head(20)
display(short_examples)
```

	metric	count	mean	median	p95	p99	max
0	char_len	9181	966.489271	864.0	1944.0	1998.0	5342
1	word_len	9181	133.112624	122.0	254.0	277.0	746
2	sent_len	9181	8.585775	8.0	14.0	18.0	38





```
     symbol                                          company_name quoteType  \
3022   MEGI  NYLI CBRE Global Infrastructure Megatrends Term Fund    EQUITY
3669   XFLT  XAI Octagon Floating Rate & Alternative Income Trust    EQUITY
4544   OCCI                              OFS Credit Company, Inc.    EQUITY
9043  BPYPM                     Brookfield Property Partners L.P.    EQUITY
9230  OCCIM                              OFS Credit Company, Inc.    EQUITY
9231  OCCIN                              OFS Credit Company, Inc.    EQUITY
9232  OCCIO                              OFS Credit Company, Inc.    EQUITY

                  sector          industry  \
3022  Financial Services  Asset Management
3669  Financial Services  Asset Management
4544  Financial Services  Asset Management
9043  Financial Services  Asset Management
9230  Financial Services  Asset Management
9231  Financial Services  Asset Management
9232  Financial Services  Asset Management


                                                                                 ␣
↪longBusinessSummary
3022  NYLI CBRE Global Infrastructure Megatrends Term Fund is headquartered in␣
↪New York, New York.
3669   XAI Octagon Floating Rate & Alternative Income Trust is headquartered in␣
↪Chicago, Illinois.
4544                                          OFS Credit Company, Inc. is a␣
↪fund of OFS Advisor.
9043                     Brookfield Property Preferred L.P. is headquartered in␣
↪Hamilton, Bermuda.
9230                                          OFS Credit Company, Inc. is a␣
↪fund of OFS Advisor.
9231                                          OFS Credit Company, Inc. is a␣
↪fund of OFS Advisor.
9232                                          OFS Credit Company, Inc. is a␣
↪fund of OFS Advisor.
```

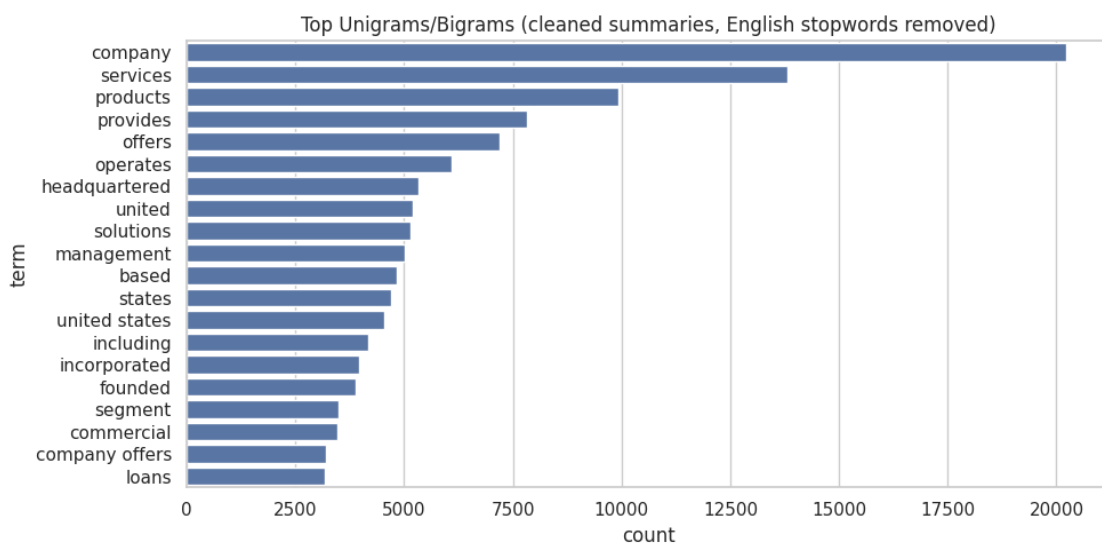
[18]:
```python
# Boilerplate phrase checks + top terms/ngrams

def basic_clean(text: str) -> str:
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

clean_texts = nonempty.map(basic_clean)
summ_lower = nonempty.str.lower()

common_phrases = [
```

```
    "headquartered in",
    "was founded in",
    "incorporated in",
    "together with its subsidiaries",
    "engages in",
    "operates in",
    "provides",
]

phrase_freq = pd.DataFrame({
    "phrase": common_phrases,
    "count": [int(summ_lower.str.contains(p).sum()) for p in common_phrases],
})
phrase_freq["pct_of_nonempty"] = phrase_freq["count"] / len(nonempty)
display(phrase_freq.sort_values("count", ascending=False))

# Top unigrams/bigrams
vec = CountVectorizer(stop_words="english", max_features=60, ngram_range=(1,2))
X = vec.fit_transform(clean_texts)
terms = np.array(vec.get_feature_names_out())
freq = np.asarray(X.sum(axis=0)).ravel()

top_idx = freq.argsort()[::-1][:30]
top_terms = pd.DataFrame({"term": terms[top_idx], "count": freq[top_idx]})
display(top_terms)

plt.figure(figsize=(10,5))
sns.barplot(data=top_terms.head(20), x="count", y="term")
plt.title("Top Unigrams/Bigrams (cleaned summaries, English stopwords removed)")
plt.tight_layout()
plt.show()
```

```
                          phrase  count  pct_of_nonempty
0                headquartered in   5341         0.581745
6                        provides   4520         0.492321
1                  was founded in   3866         0.421087
2                 incorporated in   3768         0.410413
4                      engages in   2229         0.242784
3  together with its subsidiaries   1196         0.130269
5                     operates in    806         0.087790
```

```
              term  count
0          company  20212
1         services  13808
2         products   9931
3         provides   7837
4           offers   7202
5         operates   6089
```

```
6    headquartered   5342
7           united   5208
8        solutions   5138
9       management   5011
10           based   4832
11          states   4699
12   united states   4543
13       including   4179
14    incorporated   3969
15         founded   3883
16         segment   3480
17      commercial   3453
18  company offers   3201
19           loans   3188
20        business   3163
21         systems   3005
22           known   2736
23     development   2694
24      technology   2641
25      investment   2584
26     corporation   2490
27         engages   2458
28            fund   2454
29        addition   2442
```



[19]:
```python
# Exact duplicates
summary_stripped = df[summary_col].fillna("").astype(str).str.strip()
nonempty_mask = summary_stripped != ""
```

```
dup_counts = summary_stripped[nonempty_mask].value_counts()
dup_summary_stats = pd.DataFrame([{
    "distinct_nonempty_summaries": int(dup_counts.shape[0]),
    "distinct_duplicated_summaries": int((dup_counts > 1).sum()),
    "rows_in_duplicated_summaries": int(summary_stripped[nonempty_mask].
 ↪map(dup_counts).gt(1).sum()),
    "max_repetition_single_summary": int(dup_counts.max())
}])

display(dup_summary_stats)

top_dup = dup_counts.head(10).reset_index()
top_dup.columns = ["summary_text", "count"]
top_dup["preview"] = top_dup["summary_text"].str.slice(0, 160) + "..."
display(top_dup[["count","preview"]])

# Near-duplicates with TF-IDF + NearestNeighbors
SAMPLE_N = None
rng = np.random.default_rng(42)

idx = summary_stripped[nonempty_mask].index.to_numpy()
if SAMPLE_N is not None and SAMPLE_N < len(idx):
    idx = rng.choice(idx, size=SAMPLE_N, replace=False)

texts = df.loc[idx, summary_col].fillna("").astype(str).map(basic_clean)

tfidf = TfidfVectorizer(stop_words="english", min_df=3, max_df=0.90,␣
 ↪ngram_range=(1,2))
X = tfidf.fit_transform(texts)

nn = NearestNeighbors(n_neighbors=6, metric="cosine")
nn.fit(X)
dist, neigh = nn.kneighbors(X)

SIM_THRESHOLD = 0.90
pairs = []
for row_i in range(neigh.shape[0]):
    i = idx[row_i]
    for k in range(1, neigh.shape[1]):
        j = idx[neigh[row_i, k]]
        sim = 1 - dist[row_i, k]
        if sim >= SIM_THRESHOLD:
            pairs.append((i, j, sim))

near_dup_df = (pd.DataFrame(pairs, columns=["row_i","row_j","tfidf_cosine_sim"])
               .sort_values("tfidf_cosine_sim", ascending=False))
```

```
display(near_dup_df.head(30))

# add metadata for top pairs
cols_to_show = [c for c in [symbol_col, name_col, quote_type_col, sector_col,␣
↪industry_col, summary_col] if c]
if len(near_dup_df) > 0 and cols_to_show:
    sample_pairs = near_dup_df.head(20).copy()
    left = df.loc[sample_pairs["row_i"], cols_to_show].reset_index().
↪rename(columns={"index":"row_i"})
    right = df.loc[sample_pairs["row_j"], cols_to_show].reset_index().
↪rename(columns={"index":"row_j"})
    show = sample_pairs.merge(left, on="row_i").merge(right, on="row_j",␣
↪suffixes=("_A","_B"))
    display(show.head(10))
```

```
   distinct_nonempty_summaries  distinct_duplicated_summaries  \
0                         7628                           1044

   rows_in_duplicated_summaries  max_repetition_single_summary
0                          2597                             22

   count  \
0     22
1     16
2     15
3     15
4     13
5     11
6     10
7      8
8      7
9      7


                                                                          ␣
↪                                                                         ␣
↪preview
0  Federal Home Loan Mortgage Corporation operates in the secondary mortgage␣
↪market in the United States. It operates through two segments, Single-Family␣
↪and Multi…
1  Federal National Mortgage Association provides financing solutions for␣
↪residential mortgages in the United States. The company operates through␣
↪Single-Family an…
2  Public Storage, a member of the S&P 500, is a REIT that primarily acquires,␣
↪develops, owns, and operates self-storage facilities. At September 30, 2025,␣
↪we: (i)…
3  Bank of America Corporation, through its subsidiaries, provides various␣
↪financial products and services for individual consumers, small and␣
↪middle-market busine…
```

```
4  The Connecticut Light and Power Company, a regulated electric utility,␣
↪engages in the purchase, delivery, and sale of electricity to residential,␣
↪commercial, an…
5  Enbridge Inc., together with its subsidiaries, operates as an energy␣
↪infrastructure company. The company operates through four segments: Liquids␣
↪Pipelines, Gas …
6  Morgan Stanley, a financial holding company, provides various financial␣
↪products and services to governments, financial institutions, and individuals␣
↪in the Ame…
7  Union Electric Company, doing business as Ameren Missouri, engages in the␣
↪rate-regulated electric generation, transmission, and distribution business in␣
↪Missour…
8  Adamas Trust, Inc. acquires, invests in, finances, and manages␣
↪mortgage-related single-family and multi-family residential assets in the␣
↪United States. The comp…
9  Federal Agricultural Mortgage Corporation provides a secondary market for␣
↪various loans made to borrowers in the United States. It operates through␣
↪seven segmen…
```

	row_i	row_j	tfidf_cosine_sim
4745	10283	7494	1.0
0	1	7497	1.0
4743	10276	10276	1.0
2	11	7504	1.0
3	11	7505	1.0
4	11	8365	1.0
5	11	8362	1.0
6	11	8363	1.0
4724	10219	3841	1.0
4723	10215	10215	1.0
4722	10214	10214	1.0
4721	10214	4157	1.0
4720	10213	10214	1.0
4719	10213	4157	1.0
39	77	7527	1.0
38	73	73	1.0
37	73	7579	1.0
35	66	7522	1.0
34	54	8627	1.0
33	48	7511	1.0
31	45	7523	1.0
28	40	7509	1.0
26	36	10132	1.0
25	35	7506	1.0
4706	10204	10203	1.0
4705	10204	10204	1.0
4704	10203	10203	1.0
4703	10203	10204	1.0

```
4702  10201  10201               1.0
4701  10198  10198               1.0

   row_i  row_j  tfidf_cosine_sim symbol_A        company_name_A quoteType_A  \
0  10283   7494               1.0    MTPLF       Metaplanet Inc.      EQUITY
1      1   7497               1.0    GOOGL         Alphabet Inc.      EQUITY
2  10276  10276               1.0    MAWAF  Maruwa Co., Ltd./ADR      EQUITY
3     11   7504               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
4     11   7504               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
5     11   7504               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
6     11   7504               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
7     11   7504               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
8     11   7505               1.0      JPM   JPMORGAN CHASE & CO      EQUITY
9     11   7505               1.0      JPM   JPMORGAN CHASE & CO      EQUITY

                 sector_A                     industry_A  \
0       Consumer Cyclical                    Restaurants
1  Communication Services  Internet Content & Information
2             Technology           Electronic Components
3      Financial Services             Banks - Diversified
4      Financial Services             Banks - Diversified
5      Financial Services             Banks - Diversified
6      Financial Services             Banks - Diversified
7      Financial Services             Banks - Diversified
8      Financial Services             Banks - Diversified
9      Financial Services             Banks - Diversified


 longBusinessSummary_A  \
0  Metaplanet Inc. engages in hotel management operation and development in Japan. The company operates through Hotel Business and Bitcoin Treasury Business segments. It is also i…
1  Alphabet Inc. offers various products and platforms in the United States, Europe, the Middle East, Africa, the Asia-Pacific, Canada, and Latin America. It operates through Goog…
2  Maruwa Co.,Ltd. produces and sells ceramics and electronic parts in Japan and internationally. The Company has two reportable segments: Lamic Components Business, and Lighting …
3  JPMorgan Chase & Co. operates as a financial services company worldwide. It operates through three segments: Consumer & Community Banking, Commercial & Investment Bank, and Ass…
4  JPMorgan Chase & Co. operates as a financial services company worldwide. It operates through three segments: Consumer & Community Banking, Commercial & Investment Bank, and Ass…
5  JPMorgan Chase & Co. operates as a financial services company worldwide. It operates through three segments: Consumer & Community Banking, Commercial & Investment Bank, and Ass…
```

```
6  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
7  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
8  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
9  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…

  symbol_B        company_name_B quoteType_B                sector_B  \
0    MPJPY       Metaplanet Inc.      EQUITY                     NaN
1     GOOG         Alphabet Inc.      EQUITY  Communication Services
2    MAWAF  Maruwa Co., Ltd./ADR      EQUITY              Technology
3   JPM-PC   JPMORGAN CHASE & CO      EQUITY      Financial Services
4   JPM-PC   JPMORGAN CHASE & CO      EQUITY      Financial Services
5   JPM-PC   JPMORGAN CHASE & CO      EQUITY      Financial Services
6   JPM-PC   JPMORGAN CHASE & CO      EQUITY      Financial Services
7   JPM-PC   JPMORGAN CHASE & CO      EQUITY      Financial Services
8   JPM-PD   JPMORGAN CHASE & CO      EQUITY      Financial Services
9   JPM-PD   JPMORGAN CHASE & CO      EQUITY      Financial Services

                      industry_B  \
0                            NaN
1  Internet Content & Information
2           Electronic Components
3             Banks - Diversified
4             Banks - Diversified
5             Banks - Diversified
6             Banks - Diversified
7             Banks - Diversified
8             Banks - Diversified
9             Banks - Diversified


↪                                                                                 ␣
↪                                                                                 ␣
↪ longBusinessSummary_B
0  Metaplanet Inc. engages in hotel management operation and development in␣
↪Japan. The company operates through Hotel Business and Bitcoin Treasury␣
↪Business segments. It is also i…
1  Alphabet Inc. offers various products and platforms in the United States,␣
↪Europe, the Middle East, Africa, the Asia-Pacific, Canada, and Latin America.␣
↪It operates through Goog…
```

```
2  Maruwa Co.,Ltd. produces and sells ceramics and electronic parts in Japan and␣
↪internationally. The Company has two reportable segments: Lamic Components␣
↪Business, and Lighting …
3  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
4  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
5  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
6  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
7  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
8  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
9  JPMorgan Chase & Co. operates as a financial services company worldwide. It␣
↪operates through three segments: Consumer & Community Banking, Commercial &␣
↪Investment Bank, and Ass…
```

[20]:

```python
def safe_str(x):
    if pd.isna(x):
        return ""
    return str(x).strip()

def build_embed_text(row):
    parts = []
    if safe_str(row[sector_col]):
        parts.append(f"Sector: {safe_str(row[sector_col])}")
    if safe_str(row[industry_col]):
        parts.append(f"Industry: {safe_str(row[industry_col])}")
    if safe_str(row[summary_col]):
        parts.append(f"Summary: {safe_str(row[summary_col])}")
    return " | ".join(parts).strip()

df["embed_text"] = df.apply(build_embed_text, axis=1)
df["embed_text_is_empty"] = df["embed_text"].str.strip().eq("")

embed_ready_stats = pd.DataFrame([{
    "rows_total": len(df),
    "rows_embed_text_empty": int(df["embed_text_is_empty"].sum()),
    "rows_embed_text_nonempty": int((~df["embed_text_is_empty"]).sum())
```

```
}])

display(embed_ready_stats)
```

```
   rows_total  rows_embed_text_empty  rows_embed_text_nonempty
0       10284                   1071                      9213
```

[21]:
```
def can_push():
    result = subprocess.run(
        ["git", "-C", REPO_DIR, "push", "--dry-run"],
        capture_output=True, text=True
    )
    return result.returncode == 0

def commit_and_push(msg="update"):
    subprocess.run(["git", "-C", REPO_DIR, "add", "-A"])
    subprocess.run(["git", "-C", REPO_DIR, "commit", "-m", msg])
    subprocess.run(["git", "-C", REPO_DIR, "push"])

if can_push():
    commit_and_push("update notebook / data")
else:
    print("Not a collaborator - skipping push")
```

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Everything up-to-date
```